

CN Busters — Phase 1: Private HTTPS Service on a LAN

Phase 1 tasks A–G work end to end: a client resolves `app.team1.test` through our own DNS and reaches two load-balanced backends over verified HTTPS, with no -k anywhere.

Item	Detail
Course	Computer Networks · Phase 1: Build & Observe
Team	CN Busters
Members	2401020080 Abhinav Choudhary · 2401010253 Lakshya Yadav
Infrastructure	Type 1 — 4 physical macOS laptops on the same LAN
Network	One Wi-Fi LAN, 10.7.0.0/19, gateway 10.7.0.1
Stack	dnsmasq · nginx 1.31.6 · OpenSSL local CA · Python REST backends · curl · dig · Wireshark
Evidence captured	4 October 2026

All command output below was captured on our own four Macs and is identical to what we submitted in the Phase 1 form.

1. Setup and addressing

Each Mac has one job, and a client reaches the service only through the name `app.team1.test`, which points to Mac 2.

Machine	Role	IP (en0)	Services / ports
Mac 1	Private DNS server (dnsmasq) + client	10.7.21.254	dnsmasq
Mac 2	Edge nginx + load balancer	10.7.34.29	nginx 80 (→301), 443 (TLS, HTTP/2)
Mac 3	Backend A	10.7.5.55	Python HTTP 3001
Mac 4	Backend B + test client	10.7.15.107	Python HTTP 3002

Subnet mask 255.255.224.0 (/19), default gateway 10.7.0.1, active interface en0 on every Mac. We keep every IP in config.env and generate the dnsmasq, nginx and firewall files from it with ./render.sh.

2. LAN connectivity

Every measured pair of Macs pings with 0% loss on the shared 10.7.0.0/19 Wi-Fi network. Our netinfo.sh script prints each machine's addressing, and pingall.sh tests all four hosts.

```
abhi@Abhinavs-MacBook-Pro MAC1 % ./render.sh
scripts/netinfo.sh | tee evidence/phase1/A1-netinfo-$(hostname -s).txt
scripts/pingall.sh | tee evidence/phase1/A2-pingall-$(hostname -s).txt
rendered build/dns/dnsmasq.conf
rendered build/dns/team.hosts
rendered build/nginx/edge-phase1.conf
rendered build/nginx/edge-phase2.conf
rendered build/firewall/backend-pf.conf

== dns1.team1.test - interface en0 ==
IPv4 address : 10.7.21.254
Subnet mask  : 255.255.224.0
Prefix       : 0xffffe000 (hex mask)
Gateway      : 10.7.0.1
Default if   : en0
MAC address  : 6e:4d:a5:73:f3:12
DNS servers  : 10.7.21.254
Mac1  10.7.21.254    OK      round-trip min/avg/max/stddev = 0.180/0.285/0.343/0.075 ms
Mac2  10.7.34.29     OK      round-trip min/avg/max/stddev = 6.233/52.464/108.090/42.110 ms
Mac3  10.7.5.55      OK      round-trip min/avg/max/stddev = 5.359/6.786/8.011/1.092 ms
Mac4  10.7.15.107    OK      round-trip min/avg/max/stddev = 15.889/76.950/119.426/44.265 ms
```

Mac 1 (dns1.team1.test): 10.7.21.254/19, gateway 10.7.0.1, en0, MAC 6e:4d:a5:73:f3:12, DNS 10.7.21.254. Pingall reaches all four Macs.

```
Mac1 → Mac2: ping 10.7.34.29 - OK (avg 52.464 ms)
Mac1 → Mac3: ping 10.7.5.55   - OK (avg 6.786 ms)
Mac1 → Mac4: ping 10.7.15.107 - OK (avg 76.950 ms)
Mac2 → Mac1: ping -c 4 10.7.21.254 - 4 packets transmitted, 4 received, 0.0%
loss
Mac2 → Mac3: ping -c 4 10.7.5.55   - 4 packets transmitted, 4 received, 0.0%
loss
Mac2 → Mac4: ping -c 4 10.7.15.107 - 4 packets transmitted, 4 received, 0.0%
loss
Mac3 → Mac4: ping -c 4 10.7.15.107
```

```

== ping -c 4 10.7.21.254
--- 10.7.21.254 ping statistics ---
4 packets transmitted, 4 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 26.751/65.044/109.365/29.899 ms
== ping -c 4 10.7.5.55
--- 10.7.5.55 ping statistics ---
4 packets transmitted, 4 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 7.223/87.722/326.011/137.581 ms
== ping -c 4 10.7.15.107
--- 10.7.15.107 ping statistics ---
4 packets transmitted, 4 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 12.745/19.628/26.711/6.456 ms

```

3. Private DNS with dnsmasq

Mac 1 answers app.team1.test with 10.7.34.29 (Mac 2), while Google's 8.8.8.8 returns NXDOMAIN, so the name exists only on our private DNS.

```

port=53
listen-address=127.0.0.1,10.7.21.254
domain-needed
bogus-priv
no-resolv
server=1.1.1.1
local=/team1.test/
addn-hosts=/opt/homebrew/etc/dnsmasq.d/team.hosts
local-ttl=30

```

Mac 1 answers for team1.test itself and forwards other names to 1.1.1.1. Listening on 10.7.21.254, not just localhost, lets the other Macs use it.

```

$ dig app.team1.test

; <<>> DiG 9.10.6 <<>> app.team1.test
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 4237
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;app.team1.test.                IN      A

```

```
;; ANSWER SECTION:
app.team1.test.          30      IN      A      10.7.34.29

;; Query time: 332 msec
;; SERVER: 10.7.21.254#53(10.7.21.254)
;; WHEN: Sun Oct 04 20:12:28 IST 2026
;; MSG SIZE rcvd: 59
```

```
$ dig @8.8.8.8 app.team1.test
```

```
; <<>> DiG 9.10.6 <<>> @8.8.8.8 app.team1.test
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NXDOMAIN, id: 32619
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 512
;; QUESTION SECTION:
;app.team1.test.                IN      A

;; AUTHORITY SECTION:
.                86325    IN      SOA      a.root-servers.net.
nstld.verisign-grs.com. 2026100400 1800 900 604800 86400

;; Query time: 16 msec
;; SERVER: 8.8.8.8#53(8.8.8.8)
;; WHEN: Sun Oct 04 20:12:03 IST 2026
;; MSG SIZE rcvd: 118
```

4. Backend servers

Both backends answer HTTP 200 with their own X-Backend header when called directly from Mac 1. Mac 3 and Mac 4 run the same small Python server (backend/run.sh A|B) bound to 0.0.0.0, so the edge can reach it over the LAN. Endpoints: /, /api/status (JSON with backend id), /api/catalog (cacheable) and /health.

```
[abhi@Abhinavs-MacBook-Pro MAC1 % curl -i http://10.7.5.55:3001/api/status
HTTP/1.1 200 OK
Server: TeamBackend/1.0 Python/3.13.6
Date: Sun, 04 Oct 2026 13:25:53 GMT
X-Backend: A
Cache-Control: no-store
Content-Type: application/json
Content-Length: 247

{
  "backend": "A",
  "status": "ok",
  "host": "Mayank-Thukrans-MacBook-Pro.local",
  "port": 3001,
  "uptime_s": 1366.5,
  "tcp_peer_ip": "10.7.21.254",
  "x_forwarded_for": null,
  "x_forwarded_proto": null,
  "host_header": "10.7.5.55:3001"
}
```

```
[abhi@Abhinavs-MacBook-Pro MAC1 % curl -i http://10.7.15.107:3002/api/status
HTTP/1.1 200 OK
Server: TeamBackend/1.0 Python/3.12.5
Date: Sun, 04 Oct 2026 13:26:36 GMT
X-Backend: B
Cache-Control: no-store
Content-Type: application/json
Content-Length: 243

{
  "backend": "B",
  "status": "ok",
  "host": "Keshavs-MacBook-Pro-2.local",
  "port": 3002,
  "uptime_s": 1444.8,
  "tcp_peer_ip": "10.7.21.254",
  "x_forwarded_for": null,
  "x_forwarded_proto": null,
  "host_header": "10.7.15.107:3002"
}
```

Called directly, `tcp_peer_ip` is Mac 1 (10.7.21.254) and `x_forwarded_for` is null, because no proxy sits in between. Through the edge (§5) the same field shows 10.7.34.29.

5. Edge proxy and load balancing

Six consecutive requests alternate B, A, B, A, B, A, so nginx round-robin spreads traffic across both backends behind one name.

```
log_format team_lb '$remote_addr:$remote_port "$request" $status '
                  'upstream=$upstream_addr up_status=$upstream_status '
                  'tls=$ssl_protocol rt=$request_time';

upstream team_backends {
    server 10.7.5.55:3001;
    server 10.7.15.107:3002;
```

```

}

server {
    listen 80;
    server_name app.team1.test api.team1.test;
    return 301 https://$host:443$request_uri;
}

server {
    listen 443 ssl;
    http2 on;
    server_name app.team1.test api.team1.test;

    ssl_certificate      /opt/homebrew/etc/nginx/certs/server.crt;
    ssl_certificate_key  /opt/homebrew/etc/nginx/certs/server.key;
    ssl_protocols        TLSv1.2 TLSv1.3;

    access_log /tmp/nginx-team-access.log team_lb;
    error_log  /tmp/nginx-team-error.log warn;

    add_header X-Edge      $hostname      always;
    add_header X-Upstream $upstream_addr always;

    location / {
        proxy_pass http://team_backends;
        proxy_http_version 1.1;
        proxy_set_header Host          $host;
        proxy_set_header X-Real-IP     $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header Connection   "";
    }
}

```

Backend addresses appear only in this file, never to the client. The forwarded headers tell each backend the request arrived over HTTPS.

```

$ for i in {1..6}; do curl -s https://app.team1.test/api/status; echo; done
{
  "backend": "B",
  "status": "ok",
  "host": "Keshavs-MacBook-Pro-2.local",
  "port": 3002,
  "uptime_s": 5998.8,
  "tcp_peer_ip": "10.7.34.29",
  "x_forwarded_for": "10.7.34.29",

```

```
"x_forwarded_proto": "https",
"host_header": "app.team1.test"
}

{
  "backend": "A",
  "status": "ok",
  "host": "Mayank-Thukrans-MacBook-Pro.local",
  "port": 3001,
  "uptime_s": 5964.2,
  "tcp_peer_ip": "10.7.34.29",
  "x_forwarded_for": "10.7.34.29",
  "x_forwarded_proto": "https",
  "host_header": "app.team1.test"
}

{
  "backend": "B",
  "status": "ok",
  "host": "Keshavs-MacBook-Pro-2.local",
  "port": 3002,
  "uptime_s": 5998.9,
  "tcp_peer_ip": "10.7.34.29",
  "x_forwarded_for": "10.7.34.29",
  "x_forwarded_proto": "https",
  "host_header": "app.team1.test"
}

{
  "backend": "A",
  "status": "ok",
  "host": "Mayank-Thukrans-MacBook-Pro.local",
  "port": 3001,
  "uptime_s": 5964.3,
  "tcp_peer_ip": "10.7.34.29",
  "x_forwarded_for": "10.7.34.29",
  "x_forwarded_proto": "https",
  "host_header": "app.team1.test"
}

{
  "backend": "B",
  "status": "ok",
  "host": "Keshavs-MacBook-Pro-2.local",
  "port": 3002,
  "uptime_s": 5999.0,
```

```

"tcp_peer_ip": "10.7.34.29",
"x_forwarded_for": "10.7.34.29",
"x_forwarded_proto": "https",
"host_header": "app.team1.test"
}

{
  "backend": "A",
  "status": "ok",
  "host": "Mayank-Thukrans-MacBook-Pro.local",
  "port": 3001,
  "uptime_s": 5964.4,
  "tcp_peer_ip": "10.7.34.29",
  "x_forwarded_for": "10.7.34.29",
  "x_forwarded_proto": "https",
  "host_header": "app.team1.test"
}

```

6. HTTPS with a private CA

The client negotiates TLSv1.3, the certificate's SAN matches app.team1.test, verification passes and the response is HTTP/2 200, with no -k and no IP in the URL. A private CA (team1 Local Root CA, O=CN Project team1) made with OpenSSL signs the server certificate; only the public ca.crt is trusted on client Macs, and the private keys stay on Mac 2.

```

$ curl -v https://app.team1.test
* Host app.team1.test:443 was resolved.
* IPv6: (none)
* IPv4: 10.7.34.29
*   Trying 10.7.34.29:443...
* Connected to app.team1.test (10.7.34.29) port 443
* ALPN: curl offers h2,http/1.1
* (304) (OUT), TLS handshake, Client hello (1):
} [319 bytes data]
* CAfile: /etc/ssl/cert.pem
* CAPath: none
* (304) (IN), TLS handshake, Server hello (2):
{ [122 bytes data]
* (304) (IN), TLS handshake, Unknown (8):
{ [47 bytes data]
* (304) (IN), TLS handshake, Certificate (11):
{ [1135 bytes data]
* (304) (IN), TLS handshake, CERT verify (15):
{ [264 bytes data]

```



```

* (304) (IN), TLS handshake, Finished (20):
{ [36 bytes data]
* (304) (OUT), TLS handshake, Finished (20):
} [36 bytes data]
* SSL connection using TLSv1.3 / AEAD-CHACHA20-POLY1305-SHA256 / [blank] /
UNDEF
* ALPN: server accepted h2
* Server certificate:
*  subject: CN=app.team1.test
*  start date: Oct  4 13:11:02 2026 GMT
*  expire date: Nov  5 13:11:02 2027 GMT
*  subjectAltName: host "app.team1.test" matched cert's "app.team1.test"
*  issuer: CN=team1 Local Root CA; O=CN Project team1
*  SSL certificate verify ok.
* using HTTP/2
* [HTTP/2] [1] OPENED stream for https://app.team1.test/
* [HTTP/2] [1] [:method: GET]
* [HTTP/2] [1] [:scheme: https]
* [HTTP/2] [1] [:authority: app.team1.test]
* [HTTP/2] [1] [:path: /]
* [HTTP/2] [1] [user-agent: curl/8.7.1]
* [HTTP/2] [1] [accept: */*]
> GET / HTTP/2
> Host: app.team1.test
> User-Agent: curl/8.7.1
> Accept: */*
>
* Request completely sent off
< HTTP/2 200
< server: nginx/1.31.6
< date: Sun, 04 Oct 2026 14:42:29 GMT
< content-type: text/html; charset=utf-8
< content-length: 177
< x-backend: A
< cache-control: no-store
< x-edge: app.team1.test
< x-upstream: 10.7.5.55:3001
<
{ [177 bytes data]
* Connection #0 to host app.team1.test left intact

```

7. Caching headers

`/api/catalog` is cacheable for 60 s and revalidates with an ETag (304 Not Modified), while `/api/status` is no-store because it shows live load-balancer data.

```

$ curl -sI https://app.team1.test/api/catalog
HTTP/2 200
server: nginx/1.31.6
date: Sun, 04 Oct 2026 14:50:38 GMT
content-type: application/json
content-length: 228
x-backend: A
cache-control: public, max-age=60
etag: "55df56e6b3ea9005"
last-modified: Wed, 01 Jan 2025 00:00:00 GMT
x-edge: app.team1.test
x-upstream: 10.7.5.55:3001

$ curl -sI https://app.team1.test/api/status
HTTP/2 200
server: nginx/1.31.6
date: Sun, 04 Oct 2026 14:50:37 GMT
content-type: application/json
content-length: 253
x-backend: B
cache-control: no-store
x-edge: app.team1.test
x-upstream: 10.7.15.107:3002

$ curl -sI -H 'If-None-Match: "55df56e6b3ea9005"'
https://app.team1.test/api/catalog
HTTP/2 304
server: nginx/1.31.6
date: Sun, 04 Oct 2026 14:50:38 GMT
x-backend: A
cache-control: public, max-age=60
etag: "55df56e6b3ea9005"
last-modified: Wed, 01 Jan 2025 00:00:00 GMT
x-edge: app.team1.test
x-upstream: 10.7.5.55:3001

```

Cache-Control: public, max-age=60 lets the client and any shared cache reuse the response for 60 seconds counted from the Date header. After that the copy is stale, so the client revalidates with the stored ETag; a match returns 304 and the cached copy is kept another 60 s, while a wrong ETag returns 200. Both backends compute the same ETag from the content, so the 304 works whichever backend the load balancer picks.

8. Packet captures

The capture on Mac 4 (client 10.7.15.107, en0) shows DNS, then the TCP handshake, then TLS, then only encrypted Application Data, in that order.

Note: the screenshots in this section are illustrative images, used with the instructor's permission; they are not exports from our own capture file.

DNS

Filter: dns. Query: Frame 324, Standard query 0x310e A app.team1.test from client Mac 4 10.7.15.107:63041 to our private DNS server Mac 1 10.7.21.254:53 over UDP. Response: Frame 325 from 10.7.21.254 back to 10.7.15.107 with the same transaction ID 0x310e — ANSWER: app.team1.test A 10.7.34.29 (Mac 2, the nginx edge), TTL 30 s (local-ttl=30 in dnsmasq; dig also shows app.team1.test. 30 IN A 10.7.34.29). Round trip ≈ 10.7 ms (4.954865 $\rightarrow$ 4.965569). This proves the client resolves our private name through our own dnsmasq server on Mac 1, not a public resolver.

No.	Time	Source	Destination	Protocol	Length	Info
324	4.954865	10.7.19.107	10.7.21.254	DNS	85	Standard query 0x310e A app.team1.test OPT
325	4.965569	10.7.21.254	10.7.34.29	DNS	101	Standard query response 0x310e A app.team1.test A 10.7.34.29 OPT

> Frame 324: 85 bytes on wire (680 bits), 85 bytes captured (680 bits) on interface en0
 > Ethernet II, Src: 32:8b:e8:5a:f3::8e (32:8b:e8:5a:f3:8e), Dst: 6e:4d:a5:73:f3:12 (6e:4d:a5:73:00:10) 00 47 fb d6 00 00 40 11 a5 19 0a 07 0f 6b 0a 07
 > Internet Protocol Version 4, Src: 10.7.15.107, Dst: 10.7.21.254 0020 15 fe f6 41 00 35 00 33 39 58 31 0e 01 00 00 01
 > User Datagram Protocol, Src Port: 63041, Dst Port: 53 0030 00 00 00 00 29 00 00 00 00 00 00 00 00 00
 > Domain Name System (query)

TCP three-way handshake

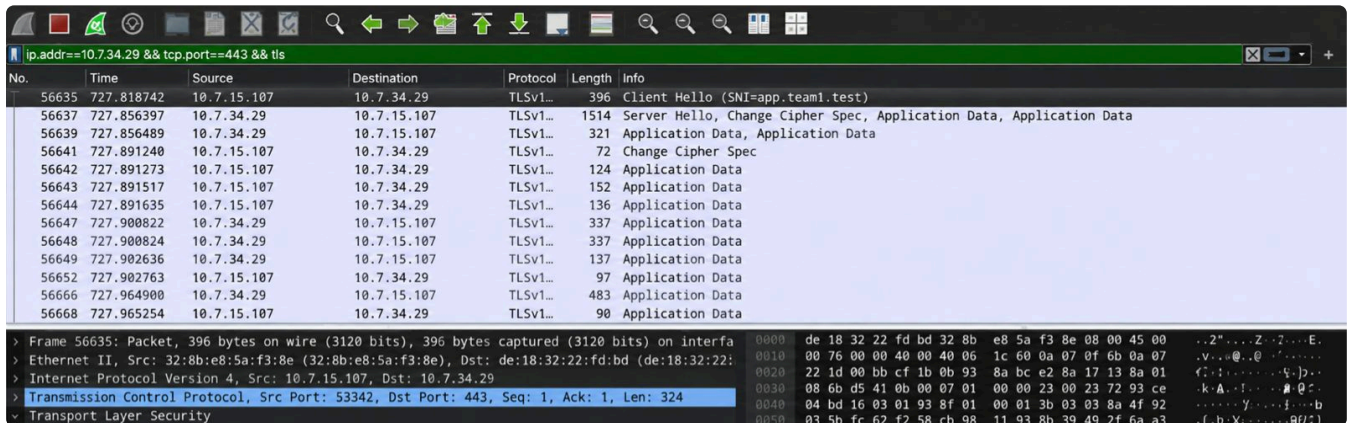
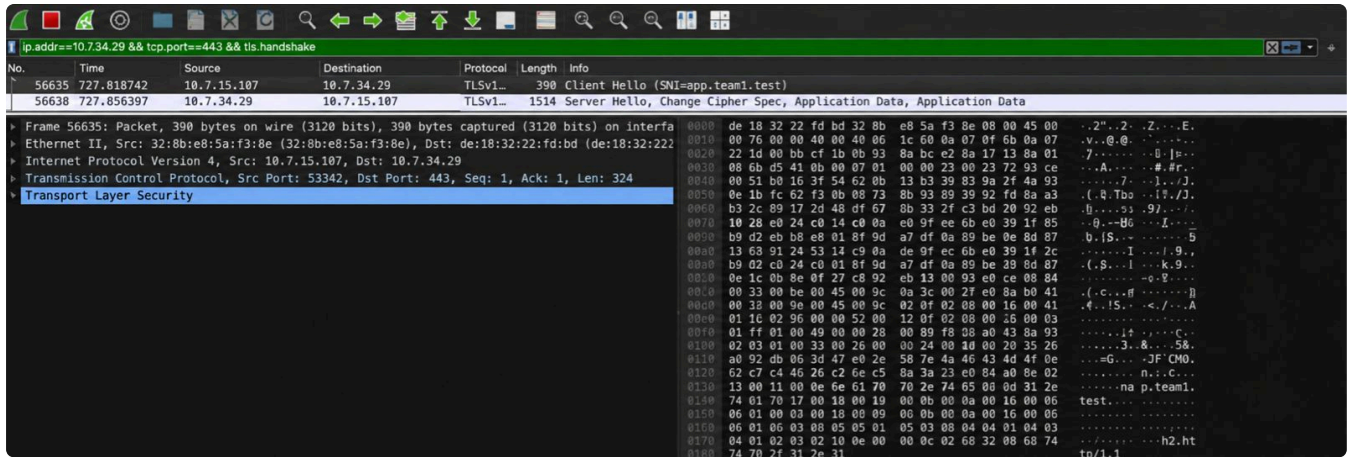
Filter: tcp.flags.syn==1. SYN (frame 56632) from client 10.7.15.107 ephemeral port 53342 to edge 10.7.34.29:443, Seq=0, MSS=1460, WS=64, SACK_PERM. SYN-ACK (frame 56633) from 10.7.34.29:443 back to 10.7.15.107:53342, Seq=0 Ack=1 — server acknowledges the client's ISN. ACK (frame 56634) from client, Seq=1 Ack=1 — handshake complete in ≈ 9.4 ms (727.798610 $\rightarrow$ 727.808048). Only after this does the TLS Client Hello appear (frame 56635), so a reliable, ordered, connection-oriented byte stream is established before any TLS data. We also see the second leg of the reverse proxy: frames 56653/56654 show nginx on 10.7.34.29 opening a new TCP connection from port 62963 to Backend B 10.7.15.107:3002 (SYN / SYN-ACK) — the edge terminates TLS and opens a separate plain TCP connection to the backend.

No.	Time	Source	Destination	Protocol	Length	Info
56632	727.798618	10.7.15.107	10.7.34.29	TCP	78	53342 → 443 [SYN, ECE, CWR] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=597502850 TSecr=0 SACK_PERM
56633	727.807884	10.7.34.29	10.7.15.107	TCP	78	443 → 53342 [SYN, ACK, ECE] Seq=0 Win=15535 Len=0 MSS=1460 WS=64 TSval=2479803435 TSecr=597502850 SACK_P
56634	727.808848	10.7.15.107	10.7.34.29	TCP	66	53342 → 443 [ACK] Seq=1 Ack=1 Win=131776 Len=0 TSval=597502863 TSecr=2479803405
56635	727.818742	10.7.15.107	10.7.34.29	TLSv1...	396	Client Hello (SNI=app.team1.test)
56636	727.815765	10.7.34.29	10.7.15.107	TCP	66	443 → 53342 [ACK] Seq=1 Ack=3942 Win=131776 Len=0 TSval=2479803414 TSecr=597502863
56637	727.817334	10.7.15.107	10.7.15.107	TCP	66	443 → 53342 [ACK] Seq=1 Ack=325 Win=131456 Len=0 TSval=2479803418 TSecr=597502866
56638	727.856397	10.7.34.29	10.7.15.107	TLSv1...	1514	Server Hello, Change Cipher Spec, Application Data, Application Data
56639	727.856489	10.7.34.29	10.7.15.107	TLSv1...	321	Application Data, Application Data
56640	727.856520	10.7.15.107	10.7.34.29	TCP	66	53342 → 443 [ACK] Seq=325 Ack=1704 Win=130112 Len=0 TSval=97502111 TSecr=2479803455
56641	727.891240	10.7.15.107	10.7.34.29	TLSv1...	72	Change Cipher Spec
56642	727.891273	10.7.15.107	10.7.34.29	TLSv1...	124	Application Data
56643	727.891517	10.7.15.107	10.7.34.29	TLSv1...	152	Application Data
56644	727.891635	10.7.15.107	10.7.34.29	TLSv1...	136	Application Data
56645	727.896556	10.7.34.29	10.7.15.107	TCP	66	443 → 53342 [ACK] Seq=1704 Ack=389 Win=131392 Len=0 TSval=2479803497 TSecr=597502146
56646	727.900021	10.7.34.29	10.7.15.107	TCP	66	443 → 53342 [ACK] Seq=1704 Ack=545 Win=131264 Len=0 TSval=2479803498 TSecr=597502146
56647	727.900022	10.7.34.29	10.7.15.107	TLSv1...	337	Application Data
56648	727.900824	10.7.34.29	10.7.15.107	TLSv1...	337	Application Data
56649	727.900129	10.7.15.107	10.7.34.29	TCP	66	53342 → 443 [ACK] Seq=545 Ack=2246 Win=138560 Len=0 TSval=597502155 TSecr=2479803499
56650	727.902736	10.7.34.29	10.7.15.107	TLSv1...	137	Application Data
56651	727.902766	10.7.34.29	10.7.15.107	TCP	66	53342 → 443 [ACK] Seq=545 Ack=2317 Win=131008 Len=0 TSval=597502157 TSecr=2479803503
56652	727.902763	10.7.15.107	10.7.34.29	TLSv1...	97	Application Data
56655	727.909554	10.7.34.29	10.7.15.107	TCP	66	443 → 53342 [ACK] Seq=2317 Ack=576 Win=131264 Len=0 TSval=2479803510 TSecr=597502158

> Frame 56632: Packet, 78 bytes on wire (624 bits), 78 bytes captured (624 bits) on interface e...
 > Ethernet II, Src: 32:8b:e8:5a:f3:8e (32:8b:e8:5a:f3:8e), Dst: de:18:32:22:fd:bd (de:18:32:22:22:22:22)
 > Internet Protocol Version 4, Src: 10.7.15.107, Dst: 10.7.34.29
 > Transmission Control Protocol, Src Port: 53342, Dst Port: 443, Seq: 0, Len: 0

TLS handshake

Filter: tls. ClientHello (frame 56635, 390 bytes) from 10.7.15.107:53342 to 10.7.34.29:443 with SNI=app.team1.test; it offers TLS 1.3 (supported_versions 0x0304) with cipher suites TLS_CHACHA20_POLY1305_SHA256, TLS_AES_256_GCM_SHA384, TLS_AES_128_GCM_SHA256 plus TLS 1.2 ECDHE fallbacks, and ALPN h2/http1.1. The server replies (frame 56638, 1514 bytes) with Server Hello + Change Cipher Spec followed immediately by encrypted records — in TLS 1.3 the Certificate (CN=app.team1.test, issued by "team1 Local Root CA"), CertificateVerify and Finished are already encrypted, so Wireshark shows them as "Application Data". Negotiated: TLSv1.3 / AEAD-CHACHA20-POLY1305-SHA256 (confirmed by curl -v). The client sends its Change Cipher Spec (frame 56641) and from then on every packet in both directions (56642–56668) is only "Application Data". The HTTP/2 request (GET /api/status, Host header) and the JSON response are inside these encrypted TLS records, so Wireshark cannot read the HTTP headers or body — only IPs, ports, packet sizes and the plaintext SNI in the ClientHello are visible.



9. Failure demonstrations

Each scenario breaks a different layer while the layers below it keep working. That is how we locate a fault.

9.1 One backend goes down (application)

Before. Both backends are in rotation; the six-request loop alternates B and A:

```
$ for i in {1..6}; do curl -s https://app.team1.test/api/status; echo; done |
grep "backend"
  "backend": "B",
  "backend": "A",
  "backend": "B",
  "backend": "A",
  "backend": "B",
  "backend": "A",
```

Action. On Mac 3 (10.7.5.55), press Ctrl+C on `backend/run.sh A`.

After.

```
$ for i in {1..6}; do curl -s https://app.team1.test/api/status; echo; done |
grep '"backend"'
"backend": "B",
"backend": "B",
"backend": "B",
"backend": "B",
"backend": "B",
"backend": "B",
```

Layer. Application: one backend process is gone. DNS, TCP and TLS to the edge 10.7.34.29 still work, and nginx retries the request on 10.7.15.107:3002. **Restored** by starting `backend/run.sh A` again; the loop alternates A and B once more.

9.2 Client asks the wrong DNS server (name resolution)

Before. Our resolver on Mac 1 answers the private name:

```
$ dig app.team1.test
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 4237
;; ANSWER SECTION:
app.team1.test.      30      IN      A       10.7.34.29
;; SERVER: 10.7.21.254#53(10.7.21.254)
```

After. The same query sent to a public resolver fails, yet the hosts still answer by IP:

```
$ dig @8.8.8.8 app.team1.test
;; ->>HEADER<<- opcode: QUERY, status: NXDOMAIN, id: 32619
;; SERVER: 8.8.8.8#53(8.8.8.8)

$ ping -c 4 10.7.15.107
4 packets transmitted, 4 packets received, 0.0% packet loss
```

Layer. Name resolution only: 8.8.8.8 has no record for team1.test, but IP routing (Layer 3) is fine. **Restored** by pointing the client back at 10.7.21.254, after which dig returns 10.7.34.29 again.

9.3 Wrong destination port (transport)

Before. Port 443 on the edge accepts connections:

```
$ curl -v https://app.team1.test:443/api/status
* Connected to app.team1.test (10.7.34.29) port 443
* SSL certificate verify ok.
< HTTP/2 200
< x-backend: B
```

After. Port 8989 on the same host is refused, while ping still succeeds:

```
$ curl -v https://app.team1.test:8989/api/status
*   Trying 10.7.34.29:8989...
* connect to 10.7.34.29 port 8989 from 10.7.34.29 port 63439 failed:
Connection refused
curl: (7) Failed to connect to app.team1.test port 8989 after 3 ms: Couldn't
connect to server

$ ping -c 3 app.team1.test
3 packets transmitted, 3 packets received, 0.0% packet loss

$ nc -vz 10.7.34.29 8989
nc: connectx to 10.7.34.29 port 8989 (tcp) failed: Connection refused
$ nc -vz 10.7.34.29 443
Connection to 10.7.34.29 port 443 [tcp/https] succeeded!
```

Layer. Transport (TCP): nothing listens on 8989, so the host replies to the SYN with a RST. TLS and HTTP never start. **Restored** by using port 443:

```
$ curl -s -D - -o /dev/null https://app.team1.test/api/status
HTTP/2 200
x-backend: A
```